SISO-UNAL: Gestión de cupos por prioridad en la inscripción de asignaturas de la Universidad Nacional de Colombia

Proyecto semestral de Estructuras de Datos — Entrega 1: propuesta, diseño e implementación inicial

Bryan Gustavo Llontop Angulo, Adrien Roho, Diego Fernando Castillo Albino
Facultad de Ingeniería, Universidad Nacional de Colombia
Sede Bogotá
Bogotá D.C., Colombia
{bllontop, aroho, dcastilloa}@unal.edu.co
Estructuras de Datos (DS3, grupo 03) — team-07 — 2026-2

***Resumen*—En la sede Bogotá de la Universidad Nacional de Colombia, cerca de 28.700 estudiantes de pregrado inscriben asignaturas mediante citas que se asignan según una prioridad institucional: los grupos de prelación y el Índice de Prioridad de Asignación (IPA). Sin embargo, esa prioridad deja de tener efecto después de la cita. No hay reserva de cupos ni lista de espera: dentro de cada franja gana quien inscribe primero, y en el periodo de adiciones y cancelaciones cada cupo liberado lo obtiene quien recargue el sistema en ese momento. El objetivo del proyecto es diseñar e implementar en Java un sistema, SISO-UNAL, que complemente las citas con tres componentes: (1) una *lista de espera por prioridad* para cada grupo, con *reasignación automática* de los cupos liberados; (2) la *sugerencia de grupos alternativos* de la misma asignatura sin cruce de horario; y (3) *reportes de demanda insatisfecha* para planear cupos. La solución usa estructuras de datos implementadas por el equipo: una cola de prioridad sobre lista enlazada ordenada para la lista de espera, una lista enlazada para los inscritos, una cola FIFO para atender la cita y una pila para el carrito con deshacer. Cada estructura se usa a través de una interfaz, para reemplazarla en las siguientes entregas por montículos, árboles balanceados, tablas hash y grafos, y comparar su desempeño. En esta primera entrega están implementadas la clasificación por prioridad, la inscripción con control de cupo, la lista de espera con aceptación previa del estudiante y la reasignación automática al cancelar.**

***Index Terms*—estructuras de datos, colas de prioridad, listas de espera, asignación de cupos, inscripción académica, análisis de complejidad**

I. Introducción y planteamiento del problema

I-A. Contexto

La sede Bogotá de la Universidad Nacional de Colombia (UNAL) tiene 32.090 estudiantes matriculados, de los cuales 28.700 son de pregrado [1]. La Facultad de Ingeniería reúne cerca del 29 % del pregrado (unos 8.320 estudiantes), seguida de Ciencias Humanas (17 %) y Ciencias (12 %). Cada estudiante inscribe entre cuatro y cinco asignaturas por semestre, es decir, entre 115.000 y 143.500 inscripciones de pregrado en cada periodo.

La inscripción se hace en el Sistema de Información Académica (SIA) mediante *citas*: franjas de 30 minutos distribuidas en tres días. Según la Circular 13 de 2026 de la Vicerrectoría Académica [2], el orden de las citas sigue cuatro grupos de prelación: (1) estudiantes en situación de discapacidad; (2) estudiantes en vulnerabilidad académica o en riesgo de abandono, por ejemplo con PAPA $\leq 3{,}1$; (3) estudiantes con un avance de al menos el 80 % de su carrera; y (4) los demás estudiantes, ordenados por el Índice de Prioridad de Asignación (IPA). El IPA tiene en cuenta el momento del plan de estudios, el rendimiento destacado, la respuesta a la encuesta docente EDIFICANDO y el Registro de Intención de Inscripción (RII).

El calendario de inscripción tiene cinco fases [3]:

1. **Consulta de la citación:** cada estudiante recibe un día y una hora según su grupo de prelación.
2. **Primera cita:** el estudiante inscribe sus asignaturas en el SIA dentro de su franja, según los cupos disponibles.
3. **Adiciones y cancelaciones (semanas 1 y 2):** se pueden agregar asignaturas si hay cupo o cancelarlas sin perder créditos.
4. **Cancelación ordinaria (semanas 3 a 8):** solo se puede cancelar, con pérdida de créditos; ya no se puede inscribir.
5. **Cancelación extemporánea (desde la semana 9):** requiere una solicitud ante el Comité Asesor y el Consejo de Facultad.

I-B. Situación que se desea resolver

Las citas deciden *cuándo* entra cada estudiante, pero después la prioridad deja de tener efecto:

- Cada franja de 30 minutos la comparten cientos de estudiantes, y dentro de ella el cupo lo obtiene quien inscribe primero.
- No hay reserva de cupos: el RII solo registra la intención del estudiante y no le garantiza nada.
- Cuando un grupo está lleno no hay lista de espera. En las adiciones y cancelaciones (fase 3), cada cupo

liberado lo obtiene quien recargue el SIA en ese momento, sin importar su prioridad.
- Si el grupo deseado está lleno, el estudiante debe buscar a mano otro grupo de la misma asignatura que no le cruce el horario.
- Los cupos que se liberan en la cancelación ordinaria (fase 4) se pierden, porque nadie puede inscribirlos.
- La universidad no sabe cuántos estudiantes quedaron sin una asignatura ni qué grupos tuvieron más demanda, información que necesita para decidir cuántos grupos abrir.

La competencia es mayor en las asignaturas que comparten varias facultades (cálculo, inglés, libre elección) y dentro de la Facultad de Ingeniería, que por su tamaño tiene su propio conflicto de cupos.

Diferencia con los sobrecupos. Cuando un grupo aparece con cupo en cero durante la inscripción o las adiciones, el estudiante puede pedir un *sobrecupo*. Según la facultad, la solicitud se hace por el módulo de solicitudes del SIA, por un formulario interno o por correo al docente. El docente o el Comité Asesor de Carrera revisa la capacidad del salón y la justificación del estudiante, y da preferencia a quien necesita la asignatura para graduarse o para no atrasarse. El sobrecupo no da créditos adicionales, y si se aprueba, la secretaría académica inscribe la asignatura a mano o habilita un cupo exclusivo.

El sobrecupo es un mecanismo complementario que no afecta directamente este proyecto. Crea cupos adicionales por excepción, con revisión manual y criterios que cambian entre facultades, mientras que SISO-UNAL se ocupa de los cupos regulares que ya existen y se liberan por cancelaciones, en orden de prioridad y sin trámite. Además, las listas de espera darían un criterio objetivo para decidir a quién conceder un sobrecupo y en qué grupos hacen falta.

I-C. Relevancia

El problema afecta a todos los estudiantes de pregrado cada semestre. Que un cupo liberado dependa de la velocidad de conexión contradice la equidad que la universidad aplica en las citas, y obliga a los estudiantes a vigilar el SIA durante días. Además, sin un registro de la demanda insatisfecha, la universidad no tiene datos para planear la apertura de grupos.

I-D. Dificultades computacionales

- **Escala:** decenas de miles de estudiantes y más de cien mil inscripciones concentradas en pocos días, por lo que repetir búsquedas lineales tiene un costo apreciable.
- **Orden por prioridad:** los estudiantes llegan a la lista de espera en cualquier orden, pero deben salir según la prioridad institucional. Esto exige una *cola de prioridad* eficiente.
- **Horarios:** un cupo liberado solo puede asignarse a un estudiante sin cruce de horario, lo que obliga a recorrer la espera hasta encontrar un candidato válido.
- **Consistencia:** un estudiante puede esperar en varios grupos de la misma asignatura y, al recibir cupo en uno, debe salir de los demás.

I-E. Propuesta general

SISO-UNAL no reemplaza las citas: las *complementa*. Mantiene la prioridad institucional y el orden de atención de la cita, y agrega tres componentes:

1. **Lista de espera por prioridad:** cada grupo lleno tiene una lista de espera ordenada por la prioridad institucional, y cada cupo liberado se asigna automáticamente al primero de la espera (Entrega 1). La verificación de cruce de horario en esa asignación se agrega en la Entrega 2.
2. **Grupos alternativos:** si el grupo deseado está lleno, el sistema sugiere otros grupos de la misma asignatura con cupo y sin cruce de horario (Entrega 2).
3. **Reportes de demanda:** los grupos y asignaturas con más estudiantes en espera o sin inscribir, para planear cupos en semestres futuros (Entregas 2 y 3).

La lista de espera es el núcleo del sistema, porque los otros dos componentes se construyen sobre ella: las sugerencias parten de quienes esperan, y los reportes se calculan con el tamaño de las esperas.

II. Objetivos

II-A. Objetivo general

Diseñar e implementar en Java un sistema que complemente las citas de inscripción de la UNAL con listas de espera por prioridad y reasignación automática de los cupos liberados, justificando las estructuras de datos elegidas para cada operación y evaluando experimentalmente su desempeño.

II-B. Objetivos específicos

1. Modelar estudiantes y grupos, y ordenar a los estudiantes según los grupos de prelación con un único método de comparación que use los datos de prioridad (incluido el IPA) como datos de entrada.
2. Atender las solicitudes de una cita en orden con una cola FIFO.
3. Permitir que el estudiante arme un carrito de selección con la opción de *deshacer*, usando una pila.
4. Implementar una lista de espera por grupo que entregue siempre al estudiante de mayor prioridad: con una lista enlazada ordenada en la Entrega 1 y con un montículo binario en la Entrega 2.
5. Reasignar automáticamente cada cupo liberado al estudiante de mayor prioridad que haya aceptado la asignación y no tenga cruce de horario.
6. Sugerir grupos alternativos de la misma asignatura, con cupo y sin cruce de horario, cuando el grupo deseado esté lleno (Entrega 2).
7. Generar reportes de demanda insatisfecha por asignatura y grupo (Entregas 2 y 3).

8. Comparar experimentalmente, con 28.700 estudiantes simulados, la asignación "gana quien recarga primero" frente a la lista de espera por prioridad, y las distintas implementaciones de cada estructura.

III. Alcance y limitaciones

III-A. Funcionalidades que se desarrollarán (semestre)

Registro de asignaturas, grupos y estudiantes; clasificación por prioridad; atención de la cita; carrito con deshacer; inscripción con control de cupo y de cruce de horario; listas de espera por prioridad; cancelación con reasignación automática durante las adiciones (fase 3); registro de los cupos liberados en la cancelación ordinaria (fase 4); búsqueda eficiente por código; sugerencia de grupos alternativos; reportes de demanda; y generación de datos simulados para los experimentos.

III-B. Fuera del alcance

Interfaz gráfica, base de datos, autenticación, conexión con el SIA real, posgrado, la asignación de las citas (fase 1; el sistema solo usa su resultado), los sobrecupos, la cancelación extemporánea (fase 5), el control de la bolsa de créditos y de la carga mínima de 10 créditos, y los criterios propios de cada facultad para estudiantes de primera matrícula.

III-C. Restricciones técnicas

Java 17 o superior, ejecución por consola y estructuras de datos implementadas por el equipo, sin usar las colecciones de `java.util`.

III-D. Supuestos

- Solo se modela el pregrado de la sede Bogotá, con datos simulados.
- Los datos de prioridad son *datos de entrada*: cada estudiante trae su condición de discapacidad, su condición de vulnerabilidad académica, su PAPA, su porcentaje de avance y su IPA, tal como los registra la universidad. **El sistema no calcula el IPA**; solo lo usa para ordenar dentro del grupo 4. Con esos datos, el sistema ubica al estudiante en su grupo de prelación.
- Orden de prioridad: primero el grupo de prelación menor. Dentro de cada grupo: en el 1, PAPA de mayor a menor; en el 2, PAPA de menor a mayor (primero quien tiene más riesgo); en el 3, avance de mayor a menor y luego PAPA; y en el 4, IPA de mayor a menor. Si hay empate, decide el código del estudiante.
- La asignación automática requiere la **aceptación previa del estudiante**. Al pedir un grupo lleno, el estudiante indica si acepta que se le inscriba automáticamente cuando se libere un cupo. Si acepta, entra a la lista de espera y, cuando sea el primero, se le inscribe sin pedirle otra confirmación, para que el cupo no quede bloqueado esperando su respuesta. Si no acepta, la solicitud se rechaza. El estudiante puede salir de la espera en cualquier momento. En la Entrega 2 se evaluará una variante en la que el cupo se ofrece al primero de la espera con un plazo para aceptarlo.
- La reasignación automática solo ocurre en las adiciones (fase 3), porque en la cancelación ordinaria (fase 4) nadie puede inscribir.
- Desde la Entrega 2, el horario de cada grupo se representará por día, hora de inicio y hora de fin.

III-E. Límites para el tiempo disponible

El proyecto avanza en tres entregas. La Entrega 1 implementa el flujo básico con estructuras lineales. Las entregas 2 y 3 reemplazan estructuras por versiones más eficientes conforme avanza el curso (montículos, árboles, tablas hash y grafos) y agregan horarios, sugerencias, reportes y experimentos.

IV. Requerimientos principales

La Tabla I presenta los requerimientos funcionales y la entrega en que se implementa cada uno.

Tabla I
Requerimientos funcionales y entrega en que se implementan

ID	Requerimiento	Entrega
RF1	Registrar los grupos de una asignatura (código y nombre de la asignatura, número de grupo y cupo).	1
RF2	Registrar estudiantes con sus datos de prioridad y ubicarlos en su grupo de prelación (el IPA es un dato de entrada).	1
RF3	Atender las solicitudes de una cita en orden con una cola FIFO.	1
RF4	Agregar grupos al carrito y deshacer el último agregado.	1
RF5	Inscribir si hay cupo y rechazar solicitudes repetidas.	1
RF6	Si el grupo está lleno y el estudiante acepta la asignación automática, ubicarlo en la lista de espera por prioridad; permitirle salir de ella.	1
RF7	Al cancelar una inscripción, asignar el cupo automáticamente al primero de la espera.	1
RF8	Mostrar los inscritos, los cupos libres y la lista de espera de un grupo.	1
RF9	Validar el cruce de horario al inscribir y al reasignar un cupo.	2
RF10	Retirar a un estudiante de las demás esperas de la misma asignatura cuando obtiene cupo.	2
RF11	Buscar estudiantes y asignaturas por código en tiempo sublineal.	2
RF12	Reportar los grupos con mayor demanda insatisfecha (top-k).	2
RF13	Sugerir otro grupo de la misma asignatura con cupo y sin cruce.	2
RF14	Validar los prerrequisitos de la malla curricular.	3
RF15	Simular las fases del calendario con 28.700 estudiantes y comparar estrategias.	3

Además, el sistema debe cumplir algunos requerimientos no funcionales: las operaciones de la lista de espera deben costar $O(\log n)$ desde la Entrega 2; el código debe compilar con un único comando documentado; el orden de prioridad

debe ser siempre el mismo para los mismos datos; y cada estructura debe poder cambiarse sin modificar las clases que la usan.

V. Trabajo relacionado inicial

Inscripción por turnos. El SIA de la UNAL asigna las citas según la prioridad institucional [2]. Atender a los estudiantes uno a uno en orden de prioridad corresponde, en la teoría de asignación, al mecanismo de *dictadura serial*, estudiado por Abdulkadiroğlu y Sönmez [4]. Este mecanismo es eficiente y difícil de manipular, pero en la UNAL solo se aplica en el momento de la cita y no maneja los cupos que se liberan después.

Asignación de cursos con preferencias. Budish y Cantillon [5] estudian la asignación de cursos por turnos de la Escuela de Negocios de Harvard y muestran que los estudiantes reportan sus preferencias de forma estratégica. Budish [6] propone un mecanismo basado en mercados, que la Escuela Wharton aplica a gran escala con el nombre de *Course Match* [7]. Estos trabajos muestran que asignar cupos es un problema computacional relevante. SISO-UNAL tiene un alcance más acotado: conserva la prioridad institucional y la extiende a los cupos que se liberan.

Listas de espera en sistemas académicos. Muchos sistemas de gestión académica tienen listas de espera que inscriben automáticamente al siguiente estudiante cuando se libera un cupo, normalmente por orden de llegada (FIFO). SISO-UNAL usa en su lugar la prioridad institucional, de acuerdo con la política de equidad de la Circular 13.

Estructuras de datos. Una cola de prioridad implementada con un montículo binario inserta y extrae el elemento de mayor prioridad en $O(\log n)$ [8]. Un árbol AVL busca en $O(\log n)$ en el peor caso, y una tabla hash, en $O(1)$ en promedio [8], [9]. Estas estructuras guían la evolución del sistema entre entregas.

VI. Diseño de la solución

VI-A. Arquitectura

El código de la Entrega 1 se organiza en dos capas y una demostración (Fig. 1):

- **Modelo:** `Student` guarda los datos de prioridad, calcula el grupo de prelación y define el orden con `compareTo`. `CourseGroup` guarda la asignatura, el número de grupo, el cupo, los inscritos y la lista de espera, y ofrece las operaciones `request`, `leaveWaitlist` y `cancel`. `RequestResult` indica el resultado de una solicitud: `ENROLLED`, `WAITLISTED` o `REJECTED`.
- **Estructuras de datos:** implementadas por el equipo y genéricas, sin `java.util`: `Node<T>`, `LinkedList<T>`, `LinkedQueue<T>`, `LinkedStack<T>`, la interfaz `PriorityQueueADT<T>` y su implementación `SortedLinkedPriorityQueue<T>`.
- **Demostración:** `Main` recorre un escenario guiado por consola.

En la Entrega 2 se agregarán una clase para los horarios con detección de cruces, un catálogo con varias asignaturas y grupos, y una clase que coordine las inscripciones de un estudiante en varios grupos.

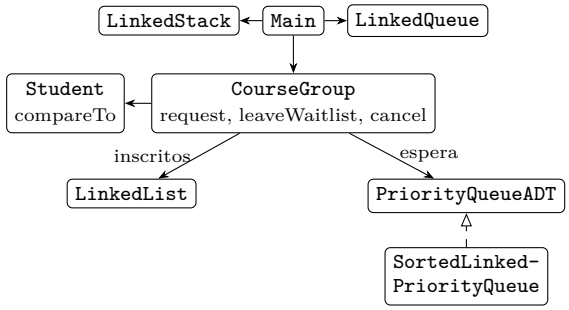


Figura 1. Clases de la Entrega 1. Las flechas sólidas indican uso; la punteada, implementación de la interfaz.

VI-B. Diseño para crecer entre entregas

La lista de espera se usa a través de una interfaz. `CourseGroup` la declara como `PriorityQueueADT<Student>` y la crea con `new SortedLinkedPriorityQueue<>()`. En la Entrega 2 basta con cambiar esa línea por un montículo binario, sin modificar `request` ni `cancel`. Tener dos implementaciones de la misma interfaz permite además compararlas con los mismos datos.

VI-C. Criterio de prioridad

Toda la regla de prioridad está en un solo método de comparación, de costo $O(1)$. Por convención, `a.compareTo(b) < 0` significa que `a` tiene *más* prioridad que `b`:

```
public int compareTo(Student other) {
  // 1) menor grupo de prelacion primero
  int byGroup = Integer.compare(getAssignedGroup(),
                                other.getAssignedGroup());
  if (byGroup != 0) return byGroup;
  // 2) criterio propio de cada grupo
  int byScore;
  switch (getAssignedGroup()) {
    case 1:  // PAPA mayor primero
      byScore = Double.compare(other.papa, papa);
      break;
    case 2:  // PAPA menor primero (mas riesgo)
      byScore = Double.compare(papa, other.papa);
      break;
    case 3:  // avance mayor primero, luego PAPA
      byScore = Double.compare(
          other.curricularProgress, curricularProgress);
      if (byScore == 0)
        byScore = Double.compare(other.papa, papa);
      break;
    default: // grupo 4: IPA mayor primero
      byScore = Double.compare(other.ipa, ipa);
  }
  if (byScore != 0) return byScore;
  // 3) desempate final: codigo del estudiante
  return code.compareTo(other.code);
}
```

El método `getAssignedGroup()` ubica al estudiante en su grupo de prelación con los datos de entrada (discapacidad; vulnerabilidad o PAPA $\leq$ 3,1; avance $\geq$ 80 %; o el resto). El IPA no se calcula: llega en el constructor de `Student`. El desempate por código garantiza que dos estudiantes nunca queden empatados.

VI-D. Flujo de información

1. Los estudiantes se insertan en una cola de prioridad, que los entrega en el orden de las citas.
2. En ese orden entran a la cola FIFO de la cita, que se atiende con `dequeue`.
3. Cada solicitud se procesa con `request` (Algoritmo 1). Si hay cupo, el estudiante queda inscrito. Si el grupo está lleno, entra a la lista de espera solo si aceptó la asignación automática; si no, la solicitud se rechaza. Una solicitud repetida también se rechaza.
4. En las adiciones, quien llega tarde a un grupo lleno queda en la espera según su prioridad, no según su hora de llegada. Con `leaveWaitlist` puede salir de la espera.
5. Al cancelar, el cupo liberado pasa automáticamente al primero de la espera (Algoritmo 2).
6. Aparte, el carrito de selección guarda grupos en una pila, y deshacer retira el último grupo agregado.

Algoritmo 1 Solicitar un grupo (`CourseGroup.request`)

Entrada: estudiante e, grupo g, aceptación a
Salida: resultado de la solicitud

```
1: if e ya está inscrito o en espera en g then ▷ O(c + w)
2:     lanzar error
3: end if
4: if g tiene cupo then
5:     agregar e al final de g.inscritos                ▷ O(1)
6:     return ENROLLED
7: else if a es verdadero then
8:     insertar e en g.espera                           ▷ O(w)
9:     return WAITLISTED
10: else
11:    return REJECTED
12: end if
```

En la Entrega 2, la reasignación recorrerá la espera hasta encontrar al primer estudiante sin cruce de horario y lo retirará de las demás esperas de la misma asignatura.

VI-E. Ejemplo de funcionamiento

La demostración de `Main` usa el grupo 1 de Cálculo Diferencial, con cupo 2, y cinco estudiantes: Ana (grupo 1, discapacidad), Carla (grupo 3, avance del 92 %), Daniel (grupo 3, avance del 85 %), Elena (grupo 4, IPA 7,5) y Felipe (grupo 4, IPA 6,0).

1. **Cita.** La cola de prioridad ordena a Ana, Carla, Elena y Felipe. Ana y Carla quedan inscritas y el grupo se llena. Elena acepta la asignación automática y entra a la espera; Felipe no la acepta y su solicitud se rechaza. Espera: [Elena].
2. **Adiciones.** Llega Daniel, que acepta la asignación automática. Aunque llegó después que Elena, queda *primero* en la espera por ser del grupo 3. Espera: [Daniel, Elena].
3. **Cancelación.** Carla cancela y el cupo pasa automáticamente a Daniel. Inscritos: [Ana, Daniel]. Espera: [Elena].
4. **Carrito.** Felipe agrega los grupos 1 y 2 a su carrito; al deshacer, se retira el grupo 2.

Algoritmo 2 Cancelar y reasignar un cupo (`CourseGroup.cancel`)

Entrada: estudiante e inscrito en el grupo g
Salida: estudiante que recibió el cupo, o nulo

```
1: if e no está en g.inscritos then
2:     lanzar error
3: end if
4: eliminar e de g.inscritos                            ▷ O(c)
5: if g.espera está vacía then
6:     return nulo
7: end if
8: p ← g.espera.removeHighest()                         ▷ O(1)
9: agregar p al final de g.inscritos                    ▷ O(1)
10: return p
```

La salida completa del programa está en el `README.md` del repositorio.

VII. Estructuras de datos seleccionadas

La Tabla II resume las estructuras de la Entrega 1, donde w es el tamaño de la lista de espera de un grupo y c su cupo. Cada estructura se incluye porque resuelve una necesidad concreta del flujo, y todas fueron implementadas por el equipo.

Costo de las operaciones del sistema. Encolar y desencolar una solicitud de la cita cuesta $O(1)$. `request` cuesta $O(c + w)$, porque primero verifica que el estudiante no esté ya inscrito ni en espera, y luego inscribe en $O(1)$ o inserta en la espera en $O(w)$. `leaveWaitlist` cuesta $O(w)$. `cancel` cuesta $O(c)$ por retirar al estudiante de los inscritos, más $O(1)$ por pasar el cupo al primero de la espera. Ordenar n estudiantes para las citas con la lista ordenada cuesta $O(n^2)$ en el peor caso; con el montículo de la Entrega 2 bajará a $O(n \log n)$.

Evolución prevista. En la Entrega 2: un montículo binario para la espera (insertar y extraer en $O(\log w)$), horarios con detección de cruces, un árbol AVL para buscar asignaturas y estudiantes por código, y un montículo para el reporte de los k grupos con más demanda ($O(n \log k)$). En la Entrega 3: una tabla hash propia para búsquedas en $O(1)$ en promedio, un grafo de prerrequisitos con recorridos BFS y DFS, y los experimentos comparativos.

Tabla II
Estructuras de datos de la Entrega 1

Estructura	Información	Operaciones	Por qué es apropiada	Alternativas	Costo
Cola de prioridad sobre lista enlazada ordenada (`SortedLinkedPriorityQueue`, interfaz `PriorityQueueADT`)	Lista de espera de cada grupo; orden de los estudiantes para las citas	insert, removeHighest, peekHighest, remove, contains	El primer nodo siempre es el de mayor prioridad, así que asignar un cupo liberado es inmediato. Se implementa con lo visto en el curso y en la Entrega 2 se reemplaza por un montículo.	Lista sin ordenar (insertar $O(1)$, extraer $O(w)$); montículo binario	insert $O(w)$; removeHighest y peekHighest $O(1)$; remove y contains $O(w)$
Lista enlazada con referencia al último nodo (`LinkedList`)	Inscritos de un grupo	addLast, remove, contains	Crece sin un límite fijo y, gracias a la referencia al último nodo, agrega al final en tiempo constante.	Arreglo dinámico (acceso por índice $O(1)$)	addLast $O(1)$; remove y contains $O(c)$
Cola FIFO enlazada (`LinkedQueue`)	Estudiantes de una cita, en orden de atención	enqueue, dequeue, peek	Atiende las solicitudes en el orden en que llegan, sin un tamaño máximo.	Cola circular sobre arreglo	$O(1)$
Pila enlazada (`LinkedStack`)	Grupos del carrito de selección	push, pop, peek	Deshacer siempre retira el grupo agregado más recientemente (LIFO).	Pila sobre arreglo dinámico	$O(1)$

VIII. Estado de la implementación

Implementado (RF1 a RF8): las estructuras genéricas `Node`, `LinkedList`, `LinkedQueue`, `LinkedStack`, `PriorityQueueADT` y `SortedLinkedPriorityQueue`; `Student`, con su grupo de prelación, el IPA como dato de entrada y `compareTo`; `CourseGroup`, con cupo, inscritos, lista de espera por prioridad con aceptación previa, salida voluntaria de la espera, rechazo de solicitudes repetidas y cancelación con reasignación automática; el carrito con deshacer; y el escenario de demostración en `Main`. El código compila con Java 17 sin advertencias y se probó en casos borde: estructuras vacías, empates, eliminaciones al inicio y al final, solicitudes repetidas y cancelaciones inválidas.

Parcialmente implementado: la reasignación funciona, pero `cancel` siempre reasigna, porque el código todavía no distingue las adiciones (fase 3) de la cancelación ordinaria (fase 4). La demostración es guiada y no tiene un menú interactivo, y trabaja con un grupo a la vez, sin catálogo. Un estudiante que espera en dos grupos de la misma asignatura todavía no sale del otro cuando obtiene cupo.

Pendiente: cruces de horario, catálogo, retiro de las demás esperas, montículo binario, árbol AVL, sugerencia de grupos y reporte de demanda (Entrega 2); tabla hash, grafo de prerrequisitos, generación de 28.700 estudiantes simulados y experimentos (Entrega 3). Los resultados, la discusión y las conclusiones se desarrollarán en esas entregas.

Repositorio: https://github.com/byepesg/DS-262, carpeta `DS3/teams/team-07/projects/entrega-01/`. El `README.md` explica cómo compilar y ejecutar, y muestra la salida esperada.

Declaración de uso de herramientas de IA

El equipo usó un asistente de inteligencia artificial (Claude, de Anthropic) para organizar la propuesta, revisar la redacción, discutir alternativas de diseño, revisar y corregir el código inicial e implementar la lista de espera por prioridad. Las decisiones, el código y su verificación son responsabilidad de los integrantes, quienes pueden explicarlos en la sustentación.

Referencias

[1] Dirección de Planeación y Estadística, Sede Bogotá, Universidad Nacional de Colombia, "Sede en cifras: Matriculados," 2026. [En línea]. Disponible en: https://planeacion.bogota.unal.edu.co/sede_en_cifras/reportes_interactivos/Matriculados/

[2] Vicerrectoría Académica, Universidad Nacional de Colombia, "Circular 13 de 2026: asignación de citas de inscripción," Bogotá, 2026.

[3] Dirección Nacional de Información Académica (DNINFOA), Universidad Nacional de Colombia, "Inscripción de asignaturas: infografía," 2026. [En línea]. Disponible en: https://dninfoa.unal.edu.co/docs/Infografia_inscripcion_asignaturas.pdf

[4] A. Abdulkadiroğlu y T. Sönmez, "Random serial dictatorship and the core from random endowments in house allocation problems," *Econometrica*, vol. 66, no. 3, pp. 689–701, 1998.

[5] E. Budish y E. Cantillon, "The multi-unit assignment problem: Theory and evidence from course allocation at Harvard," *American Economic Review*, vol. 102, no. 5, pp. 2237–2271, 2012.

[6] E. Budish, "The combinatorial assignment problem: Approximate competitive equilibrium from equal incomes," *Journal of Political Economy*, vol. 119, no. 6, pp. 1061–1103, 2011.

[7] E. Budish, G. P. Cachon, J. B. Kessler y A. Othman, "Course Match: A large-scale implementation of approximate competitive equilibrium from equal incomes for combinatorial allocation," *Operations Research*, vol. 65, no. 2, pp. 314–336, 2017.

[8] T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.

[9] M. T. Goodrich, R. Tamassia y M. H. Goldwasser, *Data Structures and Algorithms in Java*, 6th ed. Hoboken, NJ, USA: Wiley, 2014.